Opción 2: Si te refieres al LCD / DCL (*Lenguaje de Control de Datos / Data Control Language*) Wikipedia

Si por el contrario te referías a las siglas **LCD** (o *DCL*), que se emplean para gestionar la seguridad, los usuarios y los permisos de acceso a los datos, las estructuras de comandos son: TierradeLazaro

- `GRANT` : Otorgar permisos específicos a uno o varios usuarios o roles. TierradeLazaro
  - *Ejemplo:*

```sql
GRANT SELECT, INSERT ON Clientes TO Usuario1;
```

- `REVOKE` : Retirar o quitar los permisos que previamente se habían concedido con `GRANT` .
  - *Ejemplo:*

```sql
REVOKE INSERT ON Clientes FROM Usuario1;
```

- `DENY` *(disponible en gestores como SQL Server)*: Niega explícitamente un permiso a un usuario o rol, anulando cualquier otra regla heredada.
  - *Ejemplo:*

```sql
DENY DELETE ON Clientes TO Usuario1;
```

Ordre	Attribut / Clause	Rôle / Description	Exemple
1	**Nom de la colonne**	Identifiant unique de la colonne dans la table.	`id` , `email` , `date_creation`
2	**Type de données**	Définit la nature des données acceptées (texte, nombre, date...).	`INT` , `VARCHAR(255)` , `TIMESTAMP`
3	`DEFAULT`	Attribue une valeur par défaut si aucune valeur n'est spécifiée lors de l'insertion.	`DEFAULT 0` , `DEFAULT CURRENT_TIMESTAMP`
4	`NULL` / `NOT NULL`	Indique si la colonne autorise (`NULL`) ou interdit (`NOT NULL`) les valeurs vides.	`NOT NULL`
5	`PRIMARY KEY`	Désigne la colonne comme clé primaire (implique qu'elle est unique et `NOT NULL`).	`PRIMARY KEY`
6	`UNIQUE`	Garantit que deux lignes ne peuvent pas avoir la même valeur dans cette colonne.	`UNIQUE`
7	`CHECK`	Impose une condition logique que les données insérées doivent respecter.	`CHECK (age >= 18)`
8	`REFERENCES`	Établit une relation de clé étrangère vers une autre table.	`REFERENCES clients(id)`

Exemple pratique d'application

Voici comment cet ordre se traduit concrètement dans une requête `CREATE TABLE` :

```sql
CREATE TABLE commandes (
    -- 1(Nom)   2(Type)         3(Default)     4(Nullabilité) 5(Clé primaire)
    id          INT              DEFAULT 1      NOT NULL       PRIMARY KEY,

    -- 1(Nom)   2(Type)         4(Nullabilité) 6(Unique)
    reference   VARCHAR(50)      NOT NULL       UNIQUE,

    -- 1(Nom)   2(Type)         3(Default)     4(Nullabilité)
    statut      VARCHAR(20)      DEFAULT 'actif' NOT NULL,

    -- 1(Nom)   2(Type)         7(Check)
    montant     DECIMAL(10,2)    CHECK (montant > 0),

    -- 1(Nom)   2(Type)         8(References)
    client_id   INT              REFERENCES clients(id)
);
```

> **Note :** *Les contraintes multi-colonnes ou complexes (comme les clés primaires composées de plusieurs colonnes ou les clés étrangères globales) se placent généralement* **à la fin de l'instruction**, *après avoir listé toutes les colonnes de la table.*

...

El **LDM** (Lenguaje de Manipulación de Datos), conocido en inglés como **DML** (*Data Manipulation Language*), es el conjunto de instrucciones SQL que se utilizan para **consultar, insertar, modificar y eliminar** los datos almacenados dentro de las estructuras de una base de datos.

A diferencia del LDD (DDL - Lenguaje de Definición de Datos), que afecta a la estructura (crear tablas, modificar columnas), el LDM opera directamente sobre el contenido (las filas/registros).

A continuación, se detallan todas las estructuras y comandos principales del LDM en SQL:

1. Estructura principal de consulta: `SELECT`

Permite recuperar y consultar datos de una o más tablas aplicando filtros, ordenamientos y agrupamientos.

- Sintaxis general:

```sql
SELECT columnas
FROM tabla
[WHERE condiciones]
[GROUP BY columnas]
[HAVING condiciones]
[ORDER BY columnas [ASC|DESC]];
```

- Ejemplo:

```sql
SELECT nombre, email
FROM clientes
WHERE activo = 1
ORDER BY nombre ASC;
```

2. Estructura de inserción: `INSERT`

Permite añadir nuevas filas (registros) a una tabla existente.

- Sintaxis general:

```sql
-- Forma 1: Especificando las columnas (Recomendado)
INSERT INTO tabla (columna1, columna2, columna3)
VALUES (valor1, valor2, valor3);

-- Forma 2: Insertando múltiples filas a la vez
INSERT INTO tabla (columna1, columna2)
VALUES (val1_a, val2_a),
       (val1_b, val2_b);
```

- Ejemplo:

```sql
INSERT INTO productos (nombre, precio, stock)
VALUES ('Laptop Gamer', 1200.50, 15);
```

3. Estructura de actualización: `UPDATE`

Permite modificar los valores de una o varias columnas en filas ya existentes que cumplan con una condición.

> ⚠️ **Precaución:** *Si omites la cláusula* `WHERE`*, se actualizarán* **todos** *los registros de la tabla.*

3. Estructura de actualización: `UPDATE`

Permite modificar los valores de una o varias columnas en filas ya existentes que cumplan con una condición.

> ⚠️ Precaución: *Si omites la cláusula* `WHERE`*, se actualizarán* **todos** *los registros de la tabla.*

- Sintaxis general:

```sql
UPDATE tabla
SET columna1 = nuevo_valor1,
    columna2 = nuevo_valor2
[WHERE condiciones];
```

- Ejemplo:

```sql
UPDATE productos
SET precio = 1100.00, stock = stock - 1
WHERE id = 5;
```

4. Estructura de eliminación: `DELETE`

Permite borrar una o más filas de una tabla. Al igual que el `UPDATE`, requiere cuidado con el filtro `WHERE`.

- Sintaxis general:

```sql
DELETE FROM tabla
[WHERE condiciones];
```

- Ejemplo:

```sql
DELETE FROM clientes
WHERE fecha_baja < '2025-01-01';
```

5. Estructuras avanzadas / Condicionales: `MERGE` (o `UPSERT`)

Aunque no está disponible exactamente con la misma sintaxis en todos los SGBD (por ejemplo, PostgreSQL usa `INSERT ... ON CONFLICT`, MySQL usa `INSERT ... ON DUPLICATE KEY UPDATE`, y Oracle/SQL Server usan `MERGE`), se utiliza para **insertar un registro o actualizarlo automáticamente** si ya existe (basado en una clave única o primaria).

- Ejemplo genérico (PostgreSQL - UPSERT):

```sql
INSERT INTO usuarios (id, email, visitas)
VALUES (1, 'correo@example.com', 1)
ON CONFLICT (id)
DO UPDATE SET visitas = usuarios.visitas + 1;
```

Resumen Rápido de Comandos LDM

Comando	Acción Principal	Afecta a...
`SELECT`	Consultar / Leer	Los registros existentes (sin modificarlos)
`INSERT`	Crear / Insertar	Nuevas filas
`UPDATE`	Modificar / Actualizar	Filas existentes
`DELETE`	Borrar / Eliminar	Filas existentes
`MERGE` / `UPSERT`	Combinar (Insertar o Actualizar)	Filas nuevas o existentes según condición

El **Lenguaje de Definición de Datos (DDL)** en SQL es el conjunto de instrucciones utilizadas para crear, modificar, estructurar y eliminar los objetos de una base de datos (como tablas, bases de datos, índices, vistas y procedimientos).

A continuación se detallan las principales estructuras y comandos DDL junto con su sintaxis básica:

1. `CREATE` (Crear Objetos)

Se utiliza para crear nuevos objetos dentro de la base de datos desde cero.

- Crear una base de datos:

```sql
CREATE DATABASE nombre_base_datos;
```

- Crear una tabla:

```sql
CREATE TABLE usuarios (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    correo VARCHAR(100) UNIQUE,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- Crear un índice:

```SQL
CREATE INDEX idx_correo ON usuarios(correo);
```

- Crear una vista:

```SQL
CREATE VIEW vista_usuarios_activos AS
SELECT id, nombre FROM usuarios WHERE activo = 1;
```

2. `ALTER` (Modificar Estructuras)

Permite modificar la estructura de un objeto existente sin necesidad de eliminarlo ni perder sus datos (por ejemplo, agregar columnas a una tabla, cambiar tipos de datos, etc.).

- Agregar una columna a una tabla:

```SQL
ALTER TABLE usuarios ADD COLUMN fono VARCHAR(20);
```

- Eliminar una columna de una tabla:

```SQL
ALTER TABLE usuarios DROP COLUMN telefono;
```

- Modificar el tipo de dato de una columna:

```SQL
ALTER TABLE usuarios MODIFY COLUMN nombre VARCHAR(100);
```

- Agregar una restricción (Constraint):

```SQL
ALTER TABLE usuarios ADD CONSTRAINT uk_correo UNIQUE (correo);
```

3. `DROP` (Eliminar Objetos)

Elimina por completo un objeto de la base de datos, incluyendo su estructura y todos los datos que contenga. **Esta acción es irreversible.**

- Eliminar una base de datos:

```SQL
DROP DATABASE nombre_base_datos;
```

- Eliminar una tabla:

```SQL
DROP TABLE usuarios;
```

- Eliminar una tabla de forma segura (si existe):

```SQL
DROP TABLE IF EXISTS usuarios;
```

- Eliminar un índice:

```SQL
DROP INDEX idx_correo ON usuarios;
```

4. `TRUNCATE` (Vaciar Tablas)

Elimina **todos los registros** de una tabla de forma extremadamente rápida, reseteando además los contadores de autoincremento (IDs), pero **mantiene la estructura** de la tabla intacta. A diferencia de `DELETE` (que es DML), `TRUNCATE` es una operación DDL.

```sql
TRUNCATE TABLE usuarios;
```

5. `RENAME` (Renombrar Objetos)

Se utiliza para cambiarle el nombre a un objeto existente dentro de la base de datos (su sintaxis exacta puede variar ligeramente dependiendo del motor de base de datos como MySQL, PostgreSQL o SQL Server).

- **Renombrar una tabla:**

```sql
ALTER TABLE usuarios RENAME TO clientes;
```